

به نام خدا



دانشگاه تهران
دانشکده مهندسی
برق و کامپیوتر



پروژه ی درس درستی سنجی مدل های HDL

مهسا ابرغانی اقدام - ۸۱۰۱۰۳۰۵۳

مرحله چهارم طراحی واحد Golden Model

مرحله چهارم:

طراحی واحد Golden Model

- در گام های قبلی شما سناریوهایی را نوشتید که به واسطه آن بتوان عملکرد قسمت هایی از سیستم را مورد بررسی قرار داد. با تکمیل شدن این واحد، نوبت به طراحی Golden Model می رسد. در این تمرین، باید ماجولی را طراحی نمایید که با زبان ساده شما، عملکرد سیستم را شبیه سازی نماید. در این شبیه سازی باید اطلاعات تولید شده توسط Stimuli را به این واحد نیز تحویل دهید تا این واحد نیز همانند سیستم مورد بحث و همزمان با زمان بندی آن عمل نماید. در حالتی که طراحی درست باشد، خروجی های این واحد باید از دید زمان بندی و اطلاعاتی دقیقاً مشابه سیستم مورد بحث باشد. در طراحی این واحد مهم برآورده شدن نیازهای زمان بندی و عملیاتی است ولی نحوه نگارش آن می تواند با طراحی اصلی بسیار متفاوت باشد.

Golden model، ای که من نوشتم مطابق زیر است:

```
module GoldenModel (  
    input          HCLK,  
    input          HRESET,  
    input          HWRITE,  
    input          HSEL,  
    input [31:0]   HWDATA,  
    input [31:0]   HADDR,  
    output reg      HREADY,  
    output reg [31:0] HRDATA  
);
```

```

reg [31:0] memory [0:255]; // 256 locations of SDRAM
reg [3:0] state, next_state;
localparam IDLE = 4'b0000,
            READ = 4'b0001,
            WRITE = 4'b0010;

always @(posedge HCLK or posedge HRESET) begin
    if (HRESET) begin
        state <= IDLE;
        HREADY <= 1;
        HRDATA <= 32'b0;
    } else begin
        state <= next_state;
    end
end

always @(*) begin
    next_state = state;
    HREADY = 1; // Default ready state

    case (state)
    IDLE: begin
        if (HSEL && HWRITE == 0) begin
            next_state = READ;
            HREADY = 0;
        end else if (HSEL && HWRITE == 1) begin
            next_state = WRITE;
            HREADY = 0;
        end
    end

    READ: begin
        HRDATA = memory[HADDR[7:0]];
        next_state = IDLE;
    end

    WRITE: begin
        memory[HADDR[7:0]] = HWDATA;
        next_state = IDLE;
    end

    default: begin
        next_state = IDLE;
        HREADY = 1;
    end
end

```

```

        HRDATA = 32'b0;
    end

    endcase
end

endmodule

```

ماژول GoldenModel قابلیت پاسخگویی به درخواست‌های خواندن و نوشتن را دارد. این ماژول با استفاده از ماشین حالت (FSM) کار می‌کند که بین حالت‌های آماده‌باش، خواندن، و نوشتن جابجا می‌شود تا داده‌ها را از یا به حافظه منتقل کند. این ماژول یک آرایه ۲۵۶ بایتی دارد که به عنوان حافظه عمل می‌کند و با هر عملیات خواندن یا نوشتن، سیگنال‌های کنترلی مربوطه تنظیم می‌شوند. این مدل برای تست و مقایسه عملکرد دیگر اجزای سیستم به کار می‌رود.

و تست پنچ :

```

module random_stimuli_generator();

    // Testbench signals
    reg        tb_HCLK;
    reg        tb_HRESET;
    reg        tb_HWRITE;
    reg        tb_HSEL;
    reg [31:0] tb_HWDATA;
    reg [31:0] tb_HADDR;

    wire       tb_HREADY;
    wire [31:0] tb_HRDATA;
    wire       gm_HREADY;
    wire [31:0] gm_HRDATA;

    // Instance of design under test (SDRAM)
    sdram_top DUT (
        .in_HCLK(tb_HCLK),
        .in_HRESET(tb_HRESET),
        .in_HWRITE(tb_HWRITE),
        .in_HSEL(tb_HSEL),
        .in_HWDATA(tb_HWDATA),
        .in_HADDR(tb_HADDR),
        .out_HREADY(tb_HREADY),

```

```

        .out_HRDATA(tb_HRDATA)
    );

    // Instance of Golden Model
    GoldenModel golden (
        .HCLK(tb_HCLK),
        .HRESET(tb_HRESET),
        .HWRITE(tb_HWRITE),
        .HSEL(tb_HSEL),
        .HWDATA(tb_HWDATA),
        .HADDR(tb_HADDR),
        .HREADY(gm_HREADY),
        .HRDATA(gm_HRDATA)
    );

    integer file;

    // Clock generation
    initial begin
        tb_HCLK = 0;
        forever #5 tb_HCLK = ~tb_HCLK; // Generate clock
    end

    // Stimulus and scenario execution
    initial begin
        tb_HRESET = 1;
        tb_HWRITE = 0;
        tb_HSEL = 0;
        tb_HWDATA = 32'h0;
        tb_HADDR = 32'h0;

        // Open file to log results
        file = $fopen("results_comparison.txt", "w");

        // Reset system
        #100;
        tb_HRESET = 0;
        #20;

        // Scenario 1: Simple Write Operation
        @(posedge tb_HCLK);
        tb_HSEL = 1;
        tb_HWRITE = 1;
        tb_HWDATA = $random;
        tb_HADDR = $random % 256;
    end

```

```
#50;
compare_and_log(file, 1);

// Scenario 2: Simple Read Operation
@(posedge tb_HCLK);
tb_HSEL = 1;
tb_HWRITE = 0;
tb_HADDR = $random % 256;
#50;
compare_and_log(file, 2);

// Scenario 3: Write, Reset, and Read Operation
@(posedge tb_HCLK);
tb_HSEL = 1;
tb_HWRITE = 1;
tb_HWDATA = $random;
tb_HADDR = $random % 256;
#20;
tb_HRESET = 1;
#10;
tb_HRESET = 0;
#60;
tb_HSEL = 1;
tb_HWRITE = 0;
tb_HADDR = $random % 256;
#50;
compare_and_log(file, 3);

// Scenario 4: Multiple Writes
@(posedge tb_HCLK);
tb_HSEL = 1;
tb_HWRITE = 1;
tb_HWDATA = $random;
tb_HADDR = $random % 256;
#50;
compare_and_log(file, 4);

@(posedge tb_HCLK);
tb_HWDATA = $random;
tb_HADDR = $random % 256;
#50;
compare_and_log(file, 4);

// Scenario 5: Bank Boundary Test
@(posedge tb_HCLK);
```

```

tb_HSEL = 1;
tb_HWRITE = 1;
tb_HWDATA = $random;
tb_HADDR = $random % 256;
#50;
compare_and_log(file, 5);

// Scenario 6: Sequential Read After Write
@(posedge tb_HCLK);
tb_HSEL = 1;
tb_HWRITE = 1;
tb_HWDATA = $random;
tb_HADDR = $random % 256;
#50;
tb_HWRITE = 0;
tb_HADDR = $random % 256;
#50;
compare_and_log(file, 6);

// Scenario 7: Reset During Operation
@(posedge tb_HCLK);
tb_HSEL = 1;
tb_HWRITE = 1;
tb_HWDATA = $random;
tb_HADDR = $random % 256;
#30;
tb_HRESET = 1;
#20;
tb_HRESET = 0;
#50;
compare_and_log(file, 7);

$fclose(file);
$finish;
end

// Task to compare and log results from SDRAM and Golden Model
task compare_and_log(input integer file, input integer scenario_id);
begin
    $fwrite(file, "Scenario %0d: Time=%0t | ", scenario_id, $time);
    if (tb_HRDATA === gm_HRDATA && tb_HREADY === gm_HREADY) begin
        $fwrite(file, "Match: HRDATA=%h, HREADY=%b\n", tb_HRDATA,
tb_HREADY);
    end else begin

```

```

                                $fwrite(file, "Mismatch: SDRAM HRDATA=%h, HREADY=%b | Golden
HRDATA=%h, HREADY=%b\n",
                                tb_HRDATA, tb_HREADY, gm_HRDATA, gm_HREADY);
                                end
                                end
                                endtask
                                endmodule

```

در تست بنچ من یک instance از golden model ایجاد کرده و سپس در اخر کد، مقایسه ی اجرا را توسط golden model و sdram آورده ام .

خروجی:

Scenario 1: Time=175 | Mismatch: SDRAM HRDATA=xxxxxxx, HREADY=0 | Golden HRDATA=00000000, HREADY=1

Scenario 2: Time=225 | Mismatch: SDRAM HRDATA=xxxxxxx, HREADY=0 | Golden HRDATA=00000000, HREADY=0

Scenario 3: Time=365 | Mismatch: SDRAM HRDATA=xxxxxxx, HREADY=0 | Golden HRDATA=00000000, HREADY=1

Scenario 4: Time=415 | Mismatch: SDRAM HRDATA=xxxxxxx, HREADY=0 | Golden HRDATA=00000000, HREADY=0

Scenario 4: Time=465 | Mismatch: SDRAM HRDATA=xxxxxxx, HREADY=0 | Golden HRDATA=00000000, HREADY=1

Scenario 5: Time=515 | Mismatch: SDRAM HRDATA=xxxxxxx, HREADY=0 | Golden HRDATA=00000000, HREADY=0

Scenario 6: Time=615 | Mismatch: SDRAM HRDATA=xxxxxxx, HREADY=0 | Golden HRDATA=00000000, HREADY=0

Scenario 7: Time=715 | Mismatch: SDRAM HRDATA=xxxxxxx, HREADY=0 | Golden HRDATA=00000000, HREADY=1

▪ خروجی: واحد Golden Model

پیاده سازی تکنیک های تعیین پوشش Coverage

شرح: با استفاده از مفاهیم بیان شده در کلاس و نیز استفاده از ابزارهای زبان System Verilog و Modelsim در بستر طراحی شده خود، میزان پوشش ایجاد شده را بررسی و گزارش نمایید. در این تمرین دستورات استفاده شده بسیار تعیین کننده خواهد بود.

▪ خروجی: تعیین میزان Coverage Code در سیستم طراحی شده.

Coverage Details

Instance: sim:/random_sti
File: G:/uni/hdl/sdram/go
Line: 35
Statement Coverage for:
next_state
Hits: 9

Code Coverage Analysis

Statements - by instance (/random_stimuli_generator/golden)

Statement

✓

✗

E

goldenmodel.v
18 always @(posedge HCLK or posedge HRESET) begin
20 state <= IDLE;
21 HREADY <= 1;
22 HRDATA <= 32'b0;
24 state <= next_state;
28 always @(*) begin
29 next_state = state;
30 HREADY = 1; // Default ready state
35 next_state = READ;
36 HREADY = 0;
38 next_state = WRITE;
39 HREADY = 0;
44 HRDATA = memory[HADDR[7:0]];
45 next_state = IDLE;
49 memory[HADDR[7:0]] = HWDATA;
50 next_state = IDLE;
54 next_state = IDLE;
55 HREADY = 1;
56 HRDATA = 32'b0;

Analysis Assertions Cover Directives Covergroups Wave goldenmodel.v

```

M Notepad
File Edit Window
report.txt
Coverage Report Summary Data by file

=====
=== File: G:/uni/hdl/sdram/goldenmodel.v
=====
Enabled Coverage      Bins      Hits      Misses    Coverage
-----
Branches              9          9          0    100.00%
Conditions             4          3          1     75.00%
FSM States             3          3          0    100.00%
FSM Transitions        4          4          0    100.00%
Statements            19         19          0    100.00%

=====
=== File: G:/uni/hdl/sdram/sdram_controller.v
=====
Enabled Coverage      Bins      Hits      Misses    Coverage
-----
Branches             17         17          0    100.00%
Conditions             4          3          1     75.00%
FSM States            11         11          0    100.00%
FSM Transitions       20         13          7     65.00%
Statements            79         79          0    100.00%

=====
=== File: G:/uni/hdl/sdram/sdram_model.v
=====
Enabled Coverage      Bins      Hits      Misses    Coverage
-----
Branches             21         15          6     71.42%
Conditions             3          3          0    100.00%
Expressions            3          3          0    100.00%
Statements            24         20          4     83.33%

Total Coverage By File (code coverage only, filtered view): 89.43%

```

می بینیم که مقدار coverage بالاتر از ۸۰ درصد و نزدیک به ۹۰ درصد می باشد.